Sprint 14 · Reto «Proyecto final: definir, tener datos y un punto de partida»

Ficha completa del reto, para consultar. Para trabajar, el reto en una página. Qué se sube y cómo se evalúa: entrega y rúbrica.

- Volver al README del sprint.
- Si es vuestro primer reto: Cómo se trabaja un reto.
- Reglas del proyecto final (requisitos R1–R11, rúbrica de 100 puntos, anexos A y B): `briefs_y_rubrica.md`.

RETO Ficha del reto

1 Qué os pide el cliente

Qué os pide el cliente. Sois una **consultora de IA** y el docente es el **cliente y Product Owner**. En los sprints 14 a 17 vais a hacer **un proyecto completo de principio a fin**: elegir un problema, conseguir datos reales, construir modelos, evaluarlos con honestidad y defenderlo. Este es el **primer sprint de los cuatro** y en él **todavía no se entrena el mejor modelo**: se decide **qué** vais a hacer y **cómo vais a demostrar que funciona**. Lo que decidáis ahora condiciona los otros tres sprints, así que conviene pensar antes de programar. Al terminar tendréis: un problema bien definido (la **ficha de proyecto**), **datos reales con licencia**, un primer análisis de esos datos (**EDA**), un modelo de referencia muy sencillo (**baseline**), un **backlog** (lista de tareas ordenada), un **registro de riesgos éticos** y un repositorio ordenado.

Cómo se trabaja. No hay un guion con pruebas como en otros retos: **vosotros decidís** el alcance, los datos concretos y las técnicas, dentro de unos mínimos. Elegís uno de **cuatro briefs** (demanda turística, salud de cultivos con imágenes, asistente de FP con un modelo de lenguaje o demanda eléctrica) o proponéis uno propio. Os damos **plantillas** para la ficha, el plan de datos, el backlog, el registro de riesgos y el README; vuestro trabajo es rellenarlas con criterio. En clase tenéis **tres jornadas** (600 minutos de sprint): día 1, elegir brief, validar datos y entregar la ficha **antes de las 14:30**; día 2, el docente responde **go / no-go** y hacéis el EDA y el baseline; día 3, backlog, riesgos, README, entrega, review y retro. El equipo decide hasta dónde llega: mínimo, estándar o extra.

2 · Recursos del reto

Recurso	Para qué lo usaréis	Dónde está
Las reglas del proyecto: `briefs_y_rubrica.md`	Saber qué se evalúa (R1–R11), leer los cuatro briefs y la rúbrica de 100 puntos	Léelo antes del planning; subrayad lo que no entendáis
Presentación S14.1 Lanzamiento del proyecto final y sus apuntes	Aprender a formular un proyecto de IA con un *canvas* (un esquema de una página)	El docente la explica el día 1 (25 minutos)
Plantilla de la ficha `plantilla_ficha_proyecto.md`	Resumir el proyecto en una página (anexo A)	Copiadla como `ficha_proyecto.md`
Plantilla del plan de datos `plantilla_plan_datos_licencias.md`	Documentar fuente, licencia, variables, calidad, linaje y partición	Copiadla como `plan_datos_licencias.md`

Recurso	Para qué lo usaréis	Dónde está
Plantilla del backlog `plantilla_backlog_y_sprints.md`	Historias de usuario, objetivo de cada sprint y retrospectivas	Copiadla como `backlog_y_sprints.md`
Plantilla del registro de riesgos `plantilla_registro_riesgos_eticos.md`	Checklist ético, clasificación RGPD / AI Act y registro de riesgos	Copiadla como `riesgos_eticos.md`
Plantilla del README `plantilla_README_repositorio.md`	Explicar el proyecto para que otra persona lo ejecute solo con ese README	Copiadla como `README.md`. Al final lleva la sección «Revisión con la rúbrica»
Fuentes de datos candidatas (ISTAC, Frontur-Canarias, datos.gob.es, AEMET, REData, PlantVillage, catálogo oficial de FP)	Encontrar datos reales y legales	En la tabla de briefs del paso 2. **Entrad vosotros en la web y leed la licencia**
Acceso gratuito a modelos de lenguaje	Solo si elegís el brief 3 (asistente de FP)	Guía de acceso
Google Colab (o vuestro Python)	Escribir y ejecutar `01_eda.ipynb` y `02_baseline.ipynb`	Colab no necesita instalar nada. Este sprint no tiene cuadernos de práctica propios; os sirven de apoyo los de pandas y limpieza del Sprint 6 (`S06_01` y `S06_02`)
Tablero del equipo (Trello o similar)	Cargar las historias del backlog y mover las tareas	El del equipo. Si no existe, es lo primero que se crea
Cómo entregar en GitHub	Subir los ficheros a `sprint-14/`	Guía de entrega
Autoevaluación y coevaluación	Cada persona rellena su hoja	Hoja del sprint
Un asistente de IA (opcional)	Ideas, borradores y revisión	Con las reglas de la «Ayuda 1» del paso a paso

3 · Guía paso a paso

Son siete pasos, de **600 minutos en total** (200 por jornada, tres jornadas). Las jornadas de proyecto duran **6 horas completas (250 minutos de trabajo al día)**: los pasos de esta ficha ocupan unos 200 y los **50 minutos restantes de cada día son «subir de nivel»** (pasar del mínimo al estándar con la rúbrica en la mano). Cada uno dice **qué hacéis** (la lista corta), **cómo se hace** con el detalle, **qué deberíais ver** y **cuándo está hecho**. Las ayudas (IA, errores, atascos) están al final. Si el tiempo aprieta, completad el nivel mínimo y dejad lo demás para casa. **No tengáis prisa por programar:** en este sprint lo importante es pensar bien.

Paso 1 · Planning, píldora y taller (día 1, 75 minutos)

1. Leer las reglas del proyecto y repartir los roles del sprint.
2. Escuchar la píldora de teoría sobre cómo formular un proyecto de IA.
3. Hacer el taller de Design Thinking y salir con uno o dos briefs candidatos y un *canvas* borrador.

En sencillo. Hoy **no se programa**. Empieza el proyecto final: cuatro sprints en los que vais a hacer, como una consultora, un proyecto de IA entero. Antes de construir una casa se decide para quién es y qué necesita; aquí pasa igual. Hoy decidís **qué problema real vais a resolver, para quién y cómo sabréis que lo habéis resuelto**. Casi todos los proyectos de IA que fracasan lo hacen por una pregunta mal planteada, no por el modelo. Al acabar el día tendréis una hoja a lápiz (el *canvas*) y una o dos ideas candidatas. No hace falta saber Python ni IA: hace falta hablar, escuchar y anotar.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Brief	El encargo del cliente en pocas líneas: una pregunta guía y unas pistas. Hay cuatro (turismo, cultivos, asistente de FP, energía) y se puede proponer uno libre.
Cliente y Product Owner	El docente. Decide qué le sirve y responde a vuestra ficha con un *go / no-go* (seguir o ajustar).
R1 a R11	Los once requisitos que cumple todo proyecto: métrica, datos con licencia, baseline, tres modelos, una red neuronal, demostrador, repositorio…
Rúbrica de 100 puntos	La tabla con la que se puntúa el proyecto final.
Rol	Un encargo dentro del equipo. Hay cuatro: *Scrum Master*, responsable técnico, responsable de datos y responsable de IA responsable.
Rotar	Cambiar de rol en cada sprint, para que todas las personas pasen por todo.
Design Thinking	Una forma de pensar un problema empezando por las personas que lo sufren.
Canvas	Una hoja con casillas donde cabe todo el proyecto: pregunta, cliente, decisión, datos y métrica.

Cómo se hace

1. **Leer las reglas (parte de los 10 minutos de planning).** Abrid `briefs_y_rubrica.md` y leed solo dos cosas: la sección 1 (reglas comunes y la tabla de requisitos **R1 a R11**) y la sección 5 (la rúbrica de 100 puntos). Los briefs los miraréis en el paso 2. Anotad en una hoja **tres cosas que no entendáis** y preguntadlas ahora; no hace falta memorizar nada, solo saber qué se os va a pedir.
2. **Repartir los roles (el resto de los 10 minutos).** Son cuatro, y cada persona asume uno durante este sprint:
   - **Scrum Master:** cuida que el equipo trabaje bien y mantiene el tablero al día.
   - **Responsable técnico:** cuida que todo se pueda repetir (cuadernos que se ejecutan, versiones anotadas).
   - **Responsable de datos:** lleva las fuentes, las licencias y la calidad del dato.
   - **Responsable de IA responsable:** lleva los riesgos y la ética.

   Con tres personas, una asume dos roles. Los roles **rotan en cada sprint**, así que decidid ya la rotación de los sprints 15 y 16 y apuntadla en una nota del equipo; la copiaréis en la ficha del paso 4. Ejemplo con cuatro personas (nombres inventados):

Persona	Sprint 14	Sprint 15	Sprint 16
Marta	Scrum Master	Responsable técnica	Responsable de datos
Luis	Responsable técnico	Responsable de datos	Responsable de IA responsable
Ana	Responsable de datos	Responsable de IA responsable	Scrum Master
Pablo	Responsable de IA responsable	Scrum Master	Responsable técnico

3. **Escuchar la píldora (25 minutos).** El docente explica cómo se formula un proyecto de IA. Mientras tanto, cada persona anota en su hoja las cinco casillas del *canvas*: la **pregunta de negocio**, **quién usa el resultado**, la **decisión que mejora**, los **datos** y la **métrica**. Si queréis repasar luego, están los apuntes de la píldora.

4. **Hacer el taller de Design Thinking (40 minutos).** Seguid este orden:
   - **Empatía.** Pensad en un problema **real y cercano**. Preguntaos: ¿quién lo sufre?, ¿qué decisión se toma hoy sin datos?
   - **«¿Cómo podríamos…?».** Convertid el problema en preguntas abiertas, por ejemplo «¿Cómo podríamos saber con antelación qué reservas se van a cancelar?». Escribid cuantas queráis.
   - **Dos candidatos.** Quedaos con **uno o dos briefs** que encajen con vuestras preguntas (los cuatro briefs se detallan en el paso 2).
   - **Canvas borrador.** Copiad esta plantilla en papel o en una pizarra y rellenadla a lápiz para cada candidato:

```
CANVAS BORRADOR
Pregunta de negocio (una frase): ¿Podemos ... para que ...?
Quién usará el resultado:
Decisión que mejora (y qué pasa hoy sin el modelo):
Datos posibles (de dónde y si tienen licencia):
Métrica con la que sabremos que funciona:
```

   Un ejemplo bien relleno (de los apuntes de la píldora): pregunta «¿Podemos predecir qué reservas se cancelarán para que la dirección decida qué habitaciones ofrecer con descuento?»; quién usa el resultado: la directora del alojamiento; decisión: cada lunes decide qué habitaciones ofrecer con descuento y hoy lo hace sin saber qué reservas se cancelarán; métrica: de negocio, «evitar X habitaciones vacías al mes»; técnica, por ejemplo el AUC. **Prueba de la decisión:** si no sabéis decir **quién hace qué de forma distinta gracias al modelo**, todavía no tenéis proyecto: reformulad.

Qué deberíais ver. Una nota del equipo con los cuatro roles, cada uno con su nombre, y la rotación prevista. Un *canvas* (a lápiz vale) con las cinco casillas rellenas para uno o dos briefs candidatos, donde la pregunta de negocio cabe en **una frase** y nombra a quien usará el resultado.

Hecho cuando: cada persona sabe su rol y tenéis uno o dos briefs candidatos con su *canvas*.

Si algo falla

Veis…	Qué significa	Qué hacer
Somos tres personas y hay cuatro roles	Es lo previsto en tres personas.	Una persona asume dos roles; en la rotación, que no sea siempre la misma.
La pregunta dice «analizar el turismo» o «hacer algo con IA»	Es una pregunta vaga: no dice qué decisión mejora.	Aplicad la prueba de la decisión: ¿quién hace qué distinto gracias al modelo? Reescribid la pregunta con esa persona y esa decisión.
No se nos ocurre ningún problema	Falta un punto de partida.	Leed la pregunta guía de cada uno de los cuatro briefs (paso 2) y partid de la que más os llame; después adaptadla a algo cercano.
Nuestra idea necesita datos que no sabemos dónde conseguir	Sin datos accesibles no hay proyecto.	Apuntadlo en la casilla de datos como duda; se resuelve en el paso 3. Un proyecto «a la espera de datos» no se acepta.
No entendemos un requisito (R4, R6…)	Es normal el primer día.	Subrayadlo y preguntadlo en el planning; cada requisito se trabaja más adelante.
Hay dos ideas igual de buenas	Es lo que se resuelve en el paso 2.	Guardad las dos como candidatas: en el paso 2 las puntuáis y decidís.

Paso 2 · Elegir el brief (día 1, 15 minutos)

1. Comparar los briefs candidatos con tres criterios.
2. Decidir y apuntar el brief elegido, **antes del descanso**: es el primer hito del día.

En sencillo. Elegir un proyecto «a ojo» suele acabar en discusiones. Para evitarlo, hacéis lo mismo que al elegir entre dos pisos: una tabla pequeña, una nota de 1 a 5 en cada criterio y se suma. Gana el que más puntos tiene y todo el mundo sabe

por qué. No hay brief «correcto»: hay el que mejor encaja con vuestros datos, vuestro interés y el tiempo que tenéis. No hace falta saber de IA para puntuar; hace falta haber leído el paso anterior y los briefs.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Brief	El encargo del cliente: una pregunta guía y unas pistas sobre datos y técnicas.
Criterio	Cada cosa por la que se compara: aquí, datos, interés y tiempo.
Hito	Un momento del plan que hay que cumplir. El de hoy: brief elegido antes del descanso y ficha subida antes de las 14:30.
Brief libre	Un proyecto propuesto por el equipo en lugar de uno de los cuatro. Tiene condiciones.
Series temporales	Datos ordenados por fechas, como la ocupación de cada día. Los briefs 1 y 4 son de este tipo.

Cómo se hace

1. **Leer los cuatro briefs.** Esta tabla resume la pregunta de cada uno. El detalle (datos candidatos, técnicas y riesgos) está en la sección 2 de `briefs_y_rubrica.md`: leed al menos el de vuestros candidatos.

Brief	Pregunta guía	Pensad en…
1 · Demanda turística	¿Podemos predecir la llegada de turistas con 3–6 meses de antelación y qué factores la explican?	Series temporales. Carga de cómputo baja
2 · Salud de cultivos (visión)	¿Con qué fiabilidad se distingue una hoja sana de una enferma y cuándo no fiarnos?	Imágenes. Carga alta (GPU)
3 · Asistente de FP (LLM/RAG)	¿Puede un asistente responder sin inventar y cómo lo medimos?	Texto. Carga media
4 · Demanda eléctrica	¿Predecimos la demanda de las próximas 24 h mejor que un modelo de referencia?	Series temporales con calendario. Carga media

2. **Puntuar de 1 a 5 en tres criterios.** Copiad la tabla en una hoja o pizarra. Cada persona puntúa **en silencio** y después sumáis las notas de todo el equipo, brief por brief:
   - **Datos reales accesibles hoy:** 1 si no conocéis ni una fuente con licencia clara; 5 si ya sabéis dónde descargarlos.
   - **Nos interesa:** 1 si os da igual; 5 si os apetece trabajar en ello cuatro sprints.
   - **Se puede terminar en el tiempo que hay:** pensad en las unas 54 horas de proyecto. Un brief con GPU y mucho entrenamiento (el 2) o con documentos que preparar (el 3) puntúa más bajo si dudáis.

   ```
   | Brief                | Datos (1–5) | Interés (1–5) | Tiempo (1–5) | Total |
   |----------------------|-------------|---------------|--------------|-------|
   | 1 · Demanda turística  |             |               |              |       |
   | 2 · Salud de cultivos  |             |               |              |       |
   | 3 · Asistente de FP    |             |               |              |       |
   | 4 · Demanda eléctrica  |             |               |              |       |
   ```

   Ejemplo de una fila con tres personas: datos 4 + 5 + 4, interés 3 + 4 + 5, tiempo 4 + 4 + 3 → total 36. Si dos briefs empatan, desempatad con el primer criterio: sin datos no hay proyecto.

3. **Decidir y apuntar.** Marcad el brief con más puntos y escribid en la nota del equipo, en una frase, cuál es y por qué (por ejemplo, «Brief 4: es el que más puntos suma y ya sabemos dónde están los datos»). Decídselo al docente **antes del descanso**. Dos equipos pueden elegir el mismo brief, pero **cada uno define su propia pregunta y su propia métrica**: partid del *canvas* del paso 1.

4. **Brief libre (solo si queréis uno propio).** Es posible si lo presentáis en la ficha **antes de las 14:30 del día 1** y se cumple todo esto: (1) pregunta clara con métrica de éxito; (2) datos reales, accesibles y con licencia **ya identificados**; (3) cumple R1 a R11; (4) dificultad parecida a la de los otros; (5) sin datos personales sensibles. Si dudáis, elegid uno de los cuatro: no se pierde nada.

Qué deberíais ver. Una tabla con las puntuaciones de todo el equipo, un total por brief, un brief marcado como elegido y una frase que explica la elección.

Hecho cuando: el equipo ha elegido su brief, lo ha apuntado y todo el mundo sabe cuál es.

Si algo falla

Veis...	Qué significa	Qué hacer
Dos briefs tienen los mismos puntos	Empate.	Desempatad con el criterio de los datos: elegid el que tenga una fuente con licencia clara.
Cada persona quiere un brief distinto	Falta un método para decidir.	Puntuad en silencio, sumad y aceptad el resultado; es mejor un brief que a todos les parece «bien» que uno que a uno le encanta y a los demás no.
Os gusta el brief 2 pero os preocupa el tiempo de entrenamiento	Necesita GPU (se pide en Colab) y el entrenamiento tarda.	Puntuad bajo el tercer criterio y hablad con el docente antes de decidir.
Queréis un brief libre pero aún no tenéis datos	«A la espera de datos» no se acepta.	Elegid uno de los cuatro, o encontrad primero los datos (paso 3) y presentad el libre en la ficha.
Otro equipo ha elegido el mismo brief	Está permitido.	Cada equipo define su propia pregunta y su propia métrica; no copiéis la del otro.
Se acerca el descanso y no habéis decidido	El brief es el primer hito del día.	Haced la tabla de puntuación ya y decidid con el resultado; se puede afinar la pregunta después.

Paso 3 · Buscar y validar datos (día 1, 60 minutos)

1. Hacer una lista de fuentes candidatas para vuestro brief.
2. Responder, para cada fuente, a cinco preguntas de validación.
3. Rellenar `plan_datos_licencias.md` con lo que sabéis y quedaros con las fuentes que valen.

En sencillo. Un proyecto de IA vive de sus datos, y no todos los datos se pueden usar: igual que no se puede coger la foto de otra persona y ponerla en un cartel sin permiso. Este paso consiste en **buscar de dónde van a salir los datos y comprobar que son legales y utilizables**, antes de gastar horas en ellos. Se hace leyendo páginas web, no programando; solo al final hay un trozo de código opcional de cuatro líneas para comprobar que el archivo se abre. **Las licencias se leen vosotros en la web de la fuente**: una IA puede inventarse un enlace o una licencia.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Fuente	Quién publica los datos y dónde (por ejemplo, el ISTAC o datos.gob.es).
Licencia	Lo que os permite hacer con los datos. **CC BY**: usarlos citando la fuente. **CC BY-SA**: además, compartir con la misma licencia. **CC BY-NC**: solo uso no comercial. **CC0**: dominio público. Si no hay licencia clara, no se usan.
Datos personales	Cualquier dato que identifique a una persona: nombre, correo, foto con cara, ubicación exacta. Si son sensibles, no se aceptan.
Variable	Cada columna de una tabla de datos (por ejemplo, `precio_noche`).
Inventario de fuentes	La tabla del plan de datos donde se anota cada fuente con su enlace, licencia y tamaño.

Palabra	Qué significa, en sencillo
Plan de datos	El documento `plan_datos_licencias.md`: fuente, licencia, variables, calidad, linaje, partición y privacidad.

Cómo se hace

1. **Repartir la búsqueda.** El **responsable de datos** lleva la plantilla del plan de datos y las demás personas buscan y validan **una fuente cada una**. Para empezar, las candidatas de vuestro brief están en la sección 2 de `briefs_y_rubrica.md`: ISTAC, Frontur-Canarias, datos.gob.es y AEMET para el brief 1; REData y AEMET para el 4; PlantVillage para el 2; el catálogo oficial de FP para el 3. Si elegís el brief 3, mirad también la guía de acceso gratuito a modelos de lenguaje.
2. **Responder a las cinco preguntas.** Para cada fuente, **entrad en su web** y apuntad la respuesta a cada pregunta:

Pregunta	Dónde se mira	Si la respuesta es mala
¿Tiene una **licencia** que permita usarla?	En la página del conjunto de datos: buscad «Licencia», «Términos de uso» o «Aviso legal».	Si no está clara, **no se usa**. Kaggle y Hugging Face: cada conjunto tiene su licencia, no asumáis que es libre.
¿Se puede **descargar**, y podemos repetir la descarga?	Botón de descarga, enlace directo o API.	Si no se puede repetir con un enlace o un script, buscad otra fuente.
¿Qué **tamaño** tiene?	Debajo del enlace de descarga.	Si no cabe en Colab ni en vuestro ordenador, usad una parte o descartadla.
¿Qué **variables y qué periodo** trae? ¿Responden a nuestra pregunta?	Descripción del conjunto o diccionario de datos.	Si faltan las variables que necesita vuestra pregunta, cambiad de fuente o de pregunta.
¿Hay **datos personales**?	Descripción y columnas.	Si los hay y son sensibles, **no se aceptan**.

3. **Copiar la plantilla.** En GitHub, abrid `plantilla_plan_datos_licencias.md`, pulsad **Raw**, copiad todo el texto y, en la carpeta `sprint-14` del repositorio de vuestro equipo, usad *Add file → Create new file*, llamadlo `plan_datos_licencias.md`, pegad y pulsad **Commit changes** (el paso a paso con capturas está en Cómo entregar en GitHub). Hoy rellenáis tres apartados:
   - **Apartado 1, inventario de fuentes.** Una fila por fuente (D1, D2…): nombre del conjunto, enlace, fecha de descarga, licencia, si permite uso académico o redistribución, formato y tamaño, y si hay datos personales (sí / no). Ejemplo de redacción (inventado): «D1 · Ocupación diaria de un alojamiento · [enlace] · [fecha de descarga] · CC BY 4.0 · sí, citando la fuente · CSV, 60 KB · no».
   - **Apartado 2, diccionario de variables.** Una fila por columna: tipo, unidad, significado, valores válidos y porcentaje de nulos (el porcentaje de nulos lo dejáis vacío hasta el EDA del paso 5).
   - **Apartado 6, privacidad.** ¿Hay datos personales? ¿Hay datos de menores, salud o biometría? (Si la respuesta es sí, **consultad al docente antes de seguir**.) Y qué datos **no** subiréis al repositorio.

   Los apartados 3, 4 y 5 (calidad, linaje y partición) se completan el día 2 con el EDA y el baseline.

4. **(Opcional, si la fuente es un CSV) Comprobar que el archivo se abre.** En Colab, *Archivo → Cuaderno nuevo*, subid el archivo con el panel de archivos (el icono de la carpeta de la izquierda) y ejecutad en una celda:

   ```
   import pandas as pd

   datos = pd.read_csv("mis_datos.csv")
   print(datos.shape)
   print(datos.dtypes)
   print((datos.isna().mean() * 100).round(1))
   ```

   Qué hace cada línea:

   - `import pandas as pd` carga la librería que lee tablas y la apoda `pd`.
   - `pd.read_csv("mis_datos.csv")` lee el archivo y lo guarda en `datos`. **Cambiad `mis_datos.csv`** por el nombre exacto de vuestro archivo (o por la URL de descarga, entre comillas).
   - `datos.shape` da (filas, columnas), `datos.dtypes` da el tipo de cada columna y la última línea da el porcentaje de valores vacíos por columna.

   Con el conjunto de práctica `ocupacion_diaria.csv` sale `(1096, 5)`, los tipos `fecha` (texto), `ocupacion_pct`, `temperatura_c`, `festivo` y `precio_medio`, y 0.0 de nulos en todas. Con vuestros datos saldrán otras cifras: lo que importa es que **se abra** y que las columnas tengan sentido.

5. **Quedarse con las fuentes que valen.** Dejad en el inventario las que pasan las cinco preguntas. Si descartáis alguna, escribid la **razón** en su fila («descartada: licencia no indicada»). Recordad la regla de oro: **no se aceptan proyectos «a la espera de datos»**; los datos han de estar identificados y accesibles hoy.

Qué deberíais ver. Un inventario con al menos una fuente validada (enlace, licencia, tamaño, variables y «datos personales: no»), la razón escrita de cada fuente descartada y, si hicisteis el paso opcional, el tamaño de la tabla y los tipos de las columnas.

Hecho cuando: tenéis al menos una fuente con licencia clara, descargable y sin datos personales sensibles.

Si algo falla

Veis…	Qué significa	Qué hacer
La fuente no dice nada de licencia	Sin licencia no se sabe qué se permite.	Buscad en «Aviso legal» o «Términos de uso»; si sigue sin aparecer, descartadla o preguntad al docente.
La IA os da un enlace o una licencia	Las IA pueden inventarlos.	Entrad en el enlace vosotros y leed la licencia en la propia web.
`FileNotFoundError: [Errno 2] No such file or directory: 'mis_datos.csv'`	El archivo no está subido en Colab o el nombre no coincide.	Subidlo con el panel y copiad el nombre exacto, con mayúsculas y extensión.
`UnicodeDecodeError: 'utf-8' codec can't decode byte 0xf1…`	El archivo usa otra codificación (tildes y eñes).	Añadid `encoding="latin-1"`: `pd.read_csv("mis_datos.csv", encoding="latin-1")`.
`datos.shape` da `(filas, 1)` y la única columna lleva todos los nombres pegados con `;`	El archivo separa con punto y coma.	Añadid `sep=";"`: `pd.read_csv("mis_datos.csv", sep=";")`.
Los datos incluyen nombres, correos o fotos con caras	Son datos personales.	Si son sensibles, no se aceptan: buscad otra fuente o consultad al docente.
No hay forma de repetir la descarga	El dato no cumple la segunda pregunta.	Buscad otra fuente, o anotad en el plan cómo descargarla a mano y guardad la fecha.

Paso 4 · Ficha del proyecto y entrega (día 1, 50 minutos)

1. Copiar `plantilla_ficha_proyecto.md` como `ficha_proyecto.md` y rellenar los nueve apartados en una página.
2. Revisarla entre todos: ¿se entiende sin explicaciones?
3. Subirla a `sprint-14/` **antes de las 14:30**.

En sencillo. La ficha es **la hoja de una página que resume vuestro proyecto**, como el folleto de un servicio: pregunta, para quién, con qué datos y cómo sabréis si ha ido bien. El docente (vuestro cliente) la lee y os contesta *go*, *go con cambios* o *no-go*; sin ella no puede dar el visto bueno. No es un examen: se puede mejorar después, pero tiene que estar hoy. No se programa; se escribe en un archivo de texto con las plantillas que ya existen. Lo único que hay que saber es escribir con claridad.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Ficha de proyecto	El resumen de una página del proyecto (el «anexo A» de las reglas).
Markdown (`.md`)	Un archivo de texto con símbolos sencillos (`#` para los títulos, guiones para las listas). GitHub lo muestra bien formateado.
Métrica de negocio	Lo que mide el valor para el cliente, con sus palabras (euros ahorrados, horas, errores evitados).
Métrica técnica	Lo que mide cómo de bien predice el modelo (por ejemplo, MAE, F1 o AUC).
Umbral de éxito	El valor de la métrica a partir del cual diréis «el proyecto vale la pena».
Baseline	El modelo más simple posible, que habrá que superar (por ejemplo, «igual que la semana pasada»).
AI Act	El reglamento europeo de IA. Clasifica los sistemas por riesgo: mínimo, limitado, alto o inaceptable.
Commit	Cada vez que guardáis un cambio en GitHub con un mensaje corto.

Cómo se hace

1. **Copiar la plantilla.** Abrid `plantilla_ficha_proyecto.md` y pulsad **Raw**. Seleccionad todo (`Ctrl + A`) y copiad (`Ctrl + C`). En la carpeta `sprint-14` del repositorio del equipo: *Add file* → *Create new file*, nombre `ficha_proyecto.md`, pegad (`Ctrl + V`). No pulséis aún **Commit changes**: rellenad primero. El nombre, en minúsculas y sin espacios.
2. **Repartir el trabajo (sugerido).** Una persona escribe y el resto aporta, por apartados: el Scrum Master junta el texto final; el responsable de datos escribe el 4; el responsable técnico, el 5, el 6 y el 7; el responsable de IA responsable, el 8; y el 1, el 2, el 3 y el 9 se escriben entre todos. Cuidad que **todo cabe en una página**: si no cabe, resumid.

3. **Rellenar los apartados.** Sustituid los `_____` y borrad las notas en cursiva (las que van entre `*( )*` o `*…*`). Cada apartado pide una cosa:

Apartado de la ficha	Qué escribís
Cabecera	Nombre de vuestra consultora, la fecha y `Versión: 1`
1 · Equipo y roles	La tabla con las cuatro personas, su rol en el sprint 14 y la rotación prevista (copiadla de la nota del paso 1)
2 · Brief y pregunta	Dejad solo vuestro brief (1 Demanda turística · 2 Cultivos · 3 Asistente FP · 4 Energía · Libre) y la pregunta de negocio en **una frase**: «¿Podemos … para que el cliente pueda …?»
3 · Cliente y decisión	Quién usará el resultado y qué decisión concreta mejora, **y qué pasa hoy sin el modelo**
4 · Datos	Una fila por fuente validada en el paso 3: enlace, licencia, tamaño, variables clave y limitaciones previstas
5 · Métrica y baseline	Métrica de negocio, **métrica técnica**, umbral de éxito, baseline previsto y cómo partiréis los datos (temporal si son series)
6 · Modelos previstos	Al menos tres modelos, uno de ellos neuronal (o un LLM con RAG en el brief 3). Las técnicas esperadas de cada brief están en `briefs_y_rubrica.md`
7 · Demostrador	La tecnología (Streamlit, Gradio, cuaderno con widgets o API) y qué hará. El plan B ya viene escrito: un vídeo de 90 segundos
8 · Riesgos	Tres riesgos con su tipo (datos, cómputo, ética o equipo) y su mitigación; la clasificación preliminar del AI Act y si hay datos personales (sí / no)
9 · Backlog	Las historias principales (lista corta) y **una frase medible** para el objetivo de cada sprint, del 14 al 17

   Para la clasificación del AI Act, tomad esta orientación de la plantilla del plan ético: un sistema que solo predice demanda agregada suele ser de riesgo **mínimo**; un chatbot que informa a personas tiene obligaciones de **transparencia** (riesgo limitado); un sistema que decide sobre el acceso de personas a educación, empleo o servicios esenciales es de **alto riesgo**.

   Dejad la última línea (**Respuesta del docente**) como está: la rellena el docente.

4. **Un buen resultado se parece a esto** (ejemplo de redacción, no lo copiéis):
   - Pregunta: «¿Podemos predecir la ocupación hotelera de Gran Canaria con 3 meses de antelación con un error menor del 8 %, para planificar plantilla y compras?».
   - Decisión que apoya: «Cuándo contratar refuerzo en temporada media».
   - Baseline: «Predecir lo mismo que el año anterior en la misma semana».
   - Un riesgo específico: «Los datos son de hoteles grandes: no representan a pequeños alojamientos rurales».
   - Objetivo del sprint 15: «Comparar al menos tres modelos con un registro y elegir un candidato».

   La métrica, ahora. Escribid la métrica técnica y el umbral **ahora** y no los cambiéis cuando veáis los resultados: elegirlos después es la trampa más fácil de caer.

5. **Revisar entre todos (10 minutos).** Que una persona que **no escribió** la ficha la lea en voz alta como si fuera el cliente y conteste: (a) ¿la pregunta cabe en una frase?; (b) ¿dice qué decisión mejora y qué pasa hoy sin el modelo?; (c) ¿el umbral de éxito se entiende sin jerga? Si alguna respuesta es «no», corregid antes de subirla. Comprobad también que no quedan `_____` ni notas en cursiva.

6. **Subirla antes de las 14:30.** Pulsad **Commit changes**, escribid un mensaje corto («Subo la ficha de proyecto») y confirmad. El procedimiento completo, con capturas, está en Cómo entregar en GitHub. Recargad la carpeta `sprint-14` y comprobad que aparece `ficha_proyecto.md`. El docente la lee en el repositorio y responde con el *go / no-go* **el día siguiente por la mañana**.

Qué deberíais ver. Una ficha de una página, sin notas en cursiva y sin `_____`, con la fecha y `Versión: 1`; y `ficha_proyecto.md` dentro de `sprint-14/` en GitHub, con una hora de subida anterior a las 14:30 en el historial (icono del reloj).

Hecho cuando: la ficha está subida antes de las 14:30 y el docente puede abrirla.

Si algo falla

Veis...	Qué significa	Qué hacer
La ficha ocupa más de una página	Os habéis extendido.	Resumid: una frase por casilla, tablas con una o dos filas por apartado; los detalles van al plan de datos y al backlog.
No sabéis qué poner como métrica de negocio	Falta traducir el modelo a valor para el cliente.	Completad: «reducir X € de coste», «ahorrar N horas» o «evitar N errores por mes», con una cifra aproximada.
El archivo quedó en la raíz del repositorio, no en `sprint-14/`	Se subió sin entrar antes en la carpeta.	Abrid el archivo → icono del lápiz → delante del nombre escribid `sprint-14/` → **Commit changes**.
GitHub muestra las tablas como texto suelto, con barras verticales	Falta una línea en blanco antes de la tabla o se borró una barra.	Comprobad con **Preview** y volved a poner la línea en blanco o la barra que falta.
El nombre se ha subido con mayúsculas o espacios	El nombre no cumple la regla.	Renombradlo en minúsculas y sin espacios: `ficha_proyecto.md`.
Son las 14:15 y la ficha no está completa	El plazo es firme.	Subid la versión que tengáis, aunque falte algo: se corrige después del *go / no-go* con una versión 2.

Paso 5 · Go / no-go, EDA, calidad del dato y baseline (día 2, 200 minutos)

1. Leer el *go / no-go* del docente y corregir la ficha si hace falta.
2. Hacer el EDA y el informe de calidad del dato en `01_eda.ipynb`.
3. Fijar la partición de los datos y la métrica técnica **antes** de mirar resultados.
4. Construir el baseline en `02_baseline.ipynb` y calcular su métrica.

En sencillo. Hoy se mira de verdad por primera vez a los datos. Es como revisar la despensa antes de cocinar: ¿qué hay?, ¿qué falta?, ¿qué está estropeado? Eso es el **EDA** (análisis exploratorio) y su resultado es un **informe de calidad** con lo que habéis encontrado y qué hacéis con ello. Después decidís **cómo vais a repartir los datos y con qué número vais a medir**, antes de ver ningún resultado, para no engañaros. Por último, construís el **baseline**: el modelo más tonto posible, que sirve de suelo para medir las mejoras. No hace falta saber Python: copiáis el código, cambiáis las líneas que se indican y ejecutáis. Lo importante es **escribir con vuestras palabras qué significan los números**.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
EDA	Análisis exploratorio: mirar los datos (cuántos hay, qué falta, qué es raro) antes de modelar.
DataFrame (`df`)	Una tabla de datos dentro de Python. Cada fila es un caso; cada columna, una variable.
Nulo	Una casilla vacía (el dato no se registró).
Valor raro (*outlier*)	Un valor muy lejos del resto. Puede ser un error o algo real pero inusual.
Sesgo de muestreo	Que los datos no representen a todos: por ejemplo, que solo haya hoteles grandes.
Fuga de datos	Usar como pista una variable que no se conocería en el momento de decidir.
Partición	Repartir los datos en tres montones: entrenamiento (el modelo aprende), validación (se prueba y se ajusta) y test (se guarda hasta el final).
Estratificar	Repartir manteniendo en cada montón la misma proporción de cada clase.
Semilla	Un número fijo (aquí 42) que hace que el azar dé lo mismo cada vez.

Palabra	Qué significa, en sencillo
Baseline	El modelo más simple posible, que habrá que superar.

Cómo se hace

1. **Leer el *go / no-go* (20 minutos).** El docente responde con una de tres cosas: **GO** (adelante), **GO con cambios** o **NO-GO** (hay que cambiar el enfoque). Un *no-go* o un cambio es **normal**. Si pide cambios: abrid `ficha_proyecto.md` en GitHub, pulsad el icono del lápiz, corregid, cambiad `Versión: 1` por `Versión: 2` y pulsad **Commit changes** con otro mensaje («Ficha v2: ajusto el alcance»). Si el docente cambia el alcance, apuntadlo en la ficha.
2. **Crear `01_eda.ipynb` y cargar los datos (15 minutos).** En Colab: *Archivo* → *Cuaderno nuevo*, y pulsad el nombre de arriba para llamarlo `01_eda.ipynb`. Subid vuestro archivo de datos con el panel de la izquierda (o usad la URL de descarga del plan de datos). En la primera celda:

   ```
   import pandas as pd

   RUTA = "mis_datos.csv"
   SEMILLA = 42

   df = pd.read_csv(RUTA)
   print(df.shape)
   df.info()
   ```

   Qué hace cada línea:

   - `RUTA` es el nombre del archivo: **cambiadlo** por el vuestro (o por la URL entre comillas). `SEMILLA = 42` se usará para repartir los datos.
   - `pd.read_csv(RUTA)` lee la tabla y la guarda en `df`. `df.shape` da (filas, columnas) y `df.info()` lista las columnas, cuántas casillas tienen datos y su tipo.

   Con el conjunto de práctica `reservas_turisdata.csv` (sintético) sale `(6000, 18)`, y `pais_origen` tiene 5862 datos de 6000. Con los vuestros saldrán otras cifras: apuntad **filas, columnas, periodo y tipos** (resumen de tres líneas).
3. **Nulos y duplicados (20 minutos).** En una celda nueva:

   ```
   def resumen_calidad(tabla):
       return pd.DataFrame({
           "tipo": tabla.dtypes.astype(str),
           "nulos": tabla.isna().sum(),
           "pct_nulos": (tabla.isna().mean() * 100).round(2),
           "distintos": tabla.nunique(),
       })

   resumen_calidad(df)
   ```

   La función (la misma del cuaderno `S06_01`) crea una tabla con una fila por columna: su tipo, cuántos nulos tiene, qué porcentaje es y cuántos valores distintos hay. **No cambiéis los nombres.** En otra celda:

   ```
   print("Filas duplicadas:", df.duplicated().sum())
   df.describe().round(2)
   ```

   `df.duplicated().sum()` cuenta las filas repetidas exactas y `df.describe()` da, de cada columna numérica, la media, el mínimo y el máximo. Con `reservas_turisdata.csv` sale `pais_origen` con 138 nulos (2,3 %), `Filas duplicadas: 0`, `antelacion_dias` con media 89,89 y máximo 400, y `precio_noche` con media 102,87. Apuntad **cuántos hay y qué haréis** (dejarlos, quitarlos, rellenarlos), con el motivo.

4. **Valores raros (10 minutos).** Con la regla del rango intercuartílico (IQR, explicada en `S06_02`), sospechoso es lo que queda fuera de `[Q1 − 1,5·IQR, Q3 + 1,5·IQR]`:

```
COLUMNA = "precio_noche"
q1, q3 = df[COLUMNA].quantile([0.25, 0.75])
iqr = q3 - q1
limite_bajo = q1 - 1.5 * iqr
limite_alto = q3 + 1.5 * iqr
raros = df[(df[COLUMNA] < limite_bajo) | (df[COLUMNA] > limite_alto)]
print(len(raros), "valores raros en", COLUMNA, "· límites", round(limite_bajo, 1), round(limite_alto, 1))
```

   - **Cambiad `COLUMNA`** por una columna numérica vuestra (con el nombre exacto, entre comillas).
   - `quantile([0.25, 0.75])` da el valor por debajo del cual está el 25 % y el 75 % de los casos; `iqr` es la distancia entre ambos.
   - `raros` guarda las filas fuera de los límites y `|` significa «o».

   Con `precio_noche` sale `271 valores raros en precio_noche · límites 16.1 180.4`. Un valor raro no se borra sin motivo: escribid **qué son** (error o caso real) y **si los quitáis o los dejáis, y por qué**.

5. **Sesgo de muestreo y desequilibrio (10 minutos).** ¿Quién o qué falta? ¿Hay clases poco frecuentes?

```
print(df["pais_origen"].value_counts(normalize=True).round(3))
print(df["cancelada"].value_counts(normalize=True).round(3))
```

   `value_counts(normalize=True)` da la proporción de cada valor. **Cambiad** `"pais_origen"` y `"cancelada"` por vuestra columna de grupos y por la que queréis predecir. Con `reservas_turisdata.csv`: ES 0,274, UK 0,255, DE 0,148…; y `cancelada` 0 con 0,541 y 1 con 0,459. Escribid **una frase** de sesgo («solo hay hoteles grandes») y otra del desequilibrio. En el brief 2, contad las imágenes por clase (si vuestras imágenes están en una carpeta por clase):

```
import os

CARPETA = "imagenes"
cuentas = {clase: len(os.listdir(os.path.join(CARPETA, clase))) for clase in os.listdir(CARPETA)}
print(cuentas)
```

   En el brief 3, contad los documentos y su longitud con `len(texto.split())` (cuenta las palabras de un texto).

6. **Tres gráficos que respondan a vuestra pregunta (20 minutos).** Cada gráfico lleva título con la conclusión, ejes con nombre y una frase de lectura debajo:

```
import matplotlib.pyplot as plt

df["precio_noche"].hist(bins=30)
plt.title("Precio por noche de las reservas")
plt.xlabel("Precio por noche (€)")
plt.ylabel("Número de reservas")
plt.show()

df.groupby("deposito")["cancelada"].mean().plot(kind="bar")
plt.title("Las reservas sin depósito se cancelan más")
plt.xlabel("Tipo de depósito")
plt.ylabel("Proporción de reservas canceladas")
plt.show()
```

`hist(bins=30)` dibuja el histograma en 30 barras; `groupby("deposito")["cancelada"].mean()` calcula la media de `cancelada` para cada tipo de depósito; `plt.show()` muestra el dibujo. Con los datos de práctica salen las medias `parcial` 0,409, `sin_deposito` 0,540 y `total` 0,264. Para una serie temporal, el primer gráfico es la propia serie:

```
ocup = pd.read_csv(RUTA, parse_dates=["fecha"]).set_index("fecha").sort_index()
ocup["ocupacion_pct"].plot(figsize=(10, 3), title="Ocupación diaria")
plt.show()
```

7. **Fugas posibles (5 minutos, sin código).** Recorred la lista de columnas y preguntad de cada una: «¿la conoceríamos **en el momento de decidir**?». Una columna como `reembolso_emitido` solo existe **después** de cancelar: es fuga y se descarta. Escribid cuáles hay y si las descartáis.
8. **El informe de calidad (10 minutos).** Abrid `plan_datos_licencias.md` (con el icono del lápiz para editar). En el apartado 3, una fila por aspecto (nulos, duplicados, valores imposibles o raros, formatos y tipos, **sesgo de muestreo**, desequilibrio, **fuga potencial**), con el hallazgo y la acción tomada. Ejemplo (con los datos de práctica; las vuestras serán otras cifras): «Nulos: `pais_origen` 138 (2,3 %) → se dejan como una categoría más». En el apartado 4 (**linaje**) copiad la cadena «fuente → descarga → limpieza → partición → modelo» con los datos reales de cada paso. Elegid **un hallazgo que cambie el enfoque** para contarlo en la *review*.
9. **Fijar la partición y la métrica (20 minutos), antes de ejecutar nada.** Escribidlas en el apartado 5 de `plan_datos_licencias.md` y en el apartado 5 de la ficha: (a) **cómo partís**: si hay series, **por fechas** (entrenar con el pasado, evaluar con el futuro); si no, aleatoria y estratificada; y por qué eso evita fugas; (b) **tamaño** de entrenamiento, validación y test (**el test no se toca hasta que tengáis un candidato**); (c) la **métrica técnica** (MAE, MAPE, F1 o aciertos sobre 40 preguntas) y el umbral de éxito. Los tamaños los da el código siguiente: apuntadlos después.

10. **Crear `02_baseline.ipynb` y repartir los datos (25 minutos).** Cuaderno nuevo, con la misma primera celda de carga que el EDA. Para datos de tabla, con una columna a predecir:

```
from sklearn.model_selection import train_test_split

OBJETIVO = "cancelada"
X = df.drop(columns=[OBJETIVO])
y = df[OBJETIVO]

X_resto, X_test, y_resto, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=SEMILLA)
X_ent, X_val, y_ent, y_val = train_test_split(X_resto, y_resto, test_size=0.2, stratify=y_resto, random_state=SEMILLA)
print(len(X_ent), len(X_val), len(X_test))
```

    - **Cambiad `OBJETIVO`** por la columna que queréis predecir. `X` son las pistas y `y` la respuesta.
    - La primera partición aparta el 20 % para el **test**; la segunda parte el resto en entrenamiento (`_ent`) y validación (`_val`). `stratify=y` mantiene las proporciones; `random_state=SEMILLA` hace que sea siempre el mismo reparto.

    Con los datos de práctica salen `3840 960 1200`. Si vuestros datos son una **serie temporal**, no se reparte al azar sino por fechas (los últimos 91 días son 13 semanas; elegid vosotros cuántos):

```
ocup = pd.read_csv(RUTA, parse_dates=["fecha"]).set_index("fecha").sort_index()
serie = ocup["ocupacion_pct"]

DIAS = 91
entrenamiento = serie.iloc[:-2 * DIAS]
validacion = serie.iloc[-2 * DIAS:-DIAS]
test = serie.iloc[-DIAS:]
print(len(entrenamiento), len(validacion), len(test))
```

    Con `ocupacion_diaria.csv` salen `914 91 91`. `iloc` corta por posición: `[:-2 * DIAS]` es todo menos los últimos 182 días, `[-2 * DIAS:-DIAS]` los 91 anteriores a los últimos y `[-DIAS:]` los últimos 91.

11. **Construir el baseline y su métrica (35 minutos).** El modelo más simple posible. Para la tabla, «decir siempre la clase más frecuente»:

```
from sklearn.dummy import DummyClassifier
from sklearn.metrics import f1_score

baseline = DummyClassifier(strategy="most_frequent").fit(X_ent, y_ent)
pred = baseline.predict(X_val)
print("exactitud", round(baseline.score(X_val, y_val), 3), "F1", round(f1_score(y_val, pred), 3))
```

    `.fit(...)` hace que el modelo «aprenda» (aquí solo cuenta cuál es la clase más frecuente) y `.predict(...)` lo aplica a la validación. Con los datos de práctica sale `exactitud 0.541 F1 0.0`: acierta algo más de la mitad solo por ser la clase mayoritaria, y el F1 de 0 recuerda que **nunca predice la cancelación**. Para la serie, «igual que la semana pasada»:

```
from sklearn.metrics import mean_absolute_error

prediccion = serie.shift(7)
mae = mean_absolute_error(validacion, prediccion.loc[validacion.index])
print("MAE del baseline en validación:", round(mae, 2))
```

    `serie.shift(7)` desplaza la serie 7 días, de modo que cada día queda con el valor del mismo día de la semana anterior; el MAE es el error medio en puntos. Sale `MAE del baseline en validación: 5.02`. Brief 2: acierto de predecir siempre la clase más numerosa (`max(cuentas.values()) / sum(cuentas.values())`, con `cuentas` del punto 5). Brief 3: búsqueda por palabras clave sobre vuestras preguntas de referencia (`pregunta`, `respuesta_esperada`, como en `preguntas_referencia.csv`), como en el cuaderno S12_06. Escribid la frase: **«Nuestro baseline tiene [métrica] de [valor]»**. Es el suelo que superaréis en los sprints siguientes: **no lo compliquéis**.

12. **Reiniciar, descargar y subir (10 minutos).** En cada cuaderno: *Entorno de ejecución → Reiniciar y ejecutar todo*, después *Archivo → Descargar → Descargar .ipynb* y subid el fichero a `sprint-14/` (guía): cada cuaderno, una persona distinta.

Qué deberíais ver. La ficha con el *go* (o corregida); un `01_eda.ipynb` que termina con la tabla de calidad, y la partición y la métrica escritas en el plan de datos antes de ejecutar el baseline. Un `02_baseline.ipynb` que termina con un número y la frase «Nuestro baseline tiene…». Con los datos de práctica: `3840 960 1200` y `exactitud 0.541 F1 0.0` (tabla) o `914 91 91` y `MAE 5.02` (serie).

Hecho cuando: los dos cuadernos se ejecutan de arriba abajo sin errores, la métrica y la partición están escritas antes del baseline y los cuadernos están en `sprint-14/`.

Si algo falla

Veis…	Qué significa	Qué hacer
`NameError: name 'df' is not defined`	La celda de carga no se ha ejecutado, o Colab se reinició.	Ejecutad desde la primera celda hacia abajo (*Entorno de ejecución → Ejecutar todo*).
`FileNotFoundError: [Errno 2] No such file or directory: 'mis_datos.csv'`	El archivo no está subido en esta sesión de Colab, o el nombre no coincide.	Subidlo otra vez con el panel y copiad el nombre exacto en `RUTA`. Cada vez que Colab se reinicia, los archivos subidos se pierden.
`KeyError: 'cancelado'` (con vuestro nombre)	El nombre de la columna no es exacto.	Escribid `df.columns` para ver los nombres y copiadlo tal cual.
`ValueError: The least populated classes in y have only 1 member, which is too few`	Una clase tiene una sola fila y `stratify` no puede repartirla.	Quitad `stratify=y` o agrupad esa clase con otra. Apuntadlo en el informe de calidad.
El baseline da un `F1` de `0.0`	Es normal si el modelo predice siempre la clase mayoritaria.	No es un error: es justo lo que mide el baseline. Si Colab muestra un aviso naranja (*warning*) sobre «ill-defined», tampoco es un error. Citad también la exactitud y explicad que el F1 de 0 viene de que nunca predice la clase pequeña.
`UnicodeDecodeError: 'utf-8' codec can't decode byte 0xf1…`	El archivo tiene otra codificación.	`pd.read_csv(RUTA, encoding="latin-1")`; si sale una sola columna con `;`, añadid `sep=";"`.
Al *Reiniciar y ejecutar todo* una celda falla	Algo depende de una celda que está debajo, o de un archivo que no se ha vuelto a subir.	Leed el primer error en rojo, corregid esa celda y repetid hasta que todo corra.

Paso 6 · Backlog, plan ético y repositorio (día 3, 140 minutos)

1. Hacer el backlog de 10 a 15 historias y fijar el objetivo de cada sprint.
2. Rellenar el plan ético con al menos 5 riesgos específicos.
3. Montar el repositorio: README, `requirements.txt` y *commits* de todas las personas.

En sencillo. Hoy se escribe, no se programa (salvo cuatro líneas para sacar las versiones). Son tres documentos de organización. El **backlog** es la lista de tareas del proyecto, ordenada, como la lista de la compra antes de ir al súper. El **plan ético** es una revisión honesta de qué puede salir mal y a quién perjudica. El **repositorio** es la carpeta de GitHub que otra persona debería poder abrir y, con el README, repetir vuestro trabajo. Repartid el día: **cada parte la lidera una persona distinta y otra la revisa**.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Backlog	La lista ordenada de todo lo que hay que hacer en el proyecto.

Palabra	Qué significa, en sencillo
Historia de usuario	Una tarea escrita como «**Como** [rol] **quiero** [qué] **para** [beneficio]».
Criterio de aceptación	Cómo se comprueba que una historia está terminada.
Tamaño (S, M o L)	Una estimación: pequeña, mediana o grande.
Tablero	El panel (Trello o similar) con una tarjeta por historia y columnas del tipo *Por hacer / En curso / Hecho*.
Riesgo ético específico	Algo que puede salir mal **en vuestros datos y vuestro caso**. Si valdría para cualquier proyecto («cuidado con los sesgos»), es genérico.
RGPD y AI Act	La ley de protección de datos y el reglamento europeo de IA.
README	La portada del repositorio: explica el proyecto y cómo repetirlo.
`requirements.txt`	La lista de librerías con su versión, una por línea, como `pandas==2.3.3`.

Cómo se hace

1. **Backlog: copiar la plantilla (5 minutos).** Copiad `plantilla_backlog_y_sprints.md` como `backlog_y_sprints.md` en `sprint-14` (**Raw** → copiar → *Create new file*, como en el paso 4). Escribid el nombre del proyecto y quién es el Scrum Master de este sprint.
2. **Backlog: adaptar las historias (25 minutos).** La tabla del apartado 2 trae doce historias de ejemplo (H1 a H12). **Adaptadlas** a vuestro proyecto y añadid las vuestras (H13…) hasta tener **entre 10 y 15**. Cada fila lleva: historia, criterios de aceptación, tamaño (S, M o L), sprint (14 a 17), responsable y estado. Si no sabéis cómo comprobar una historia, no está bien definida. Una historia = una tarea que cabe en unas horas. Ejemplo para el brief 4:

   ```
   | H13 | Como cliente quiero ver el error por franja horaria para saber cuándo no fiarme de la previsión | Gráfico del error por hora en el demostrador | M | 16 | (responsable) | por hacer |
   ```

3. **Backlog: objetivos de sprint y tablero (15 minutos).** En el apartado 3 escribid, para los sprints 14, 15, 16 y 17, **una frase medible** en la columna «Objetivo» (no toquéis las columnas de fechas ni de hitos, que ya vienen). Ejemplos: sprint 15, «tres modelos comparados con un registro y un candidato elegido»; sprint 16, «demostrador, evaluación, análisis de sesgos y memoria». Después cargad las historias en el **tablero**: una tarjeta por historia. Si no existe tablero, es lo primero que se crea. El **Scrum Master** lo mantiene al día. El apartado 5 (retrospectivas) se rellena en el paso 7.

4. **Plan ético: copiar y rellenar (45 minutos).** Copiad `plantilla_registro_riesgos_eticos.md` como `riesgos_eticos.md`; en la línea «Versión vigente» dejad solo S14. Este documento **actualiza el checklist del Sprint 1** (`checklist_etico_v1.md`): si lo tenéis, pasad vuestras respuestas a la columna S1. Tiene cuatro partes:
   - **Parte A (checklist, seis bloques A1 a A6).** Rellenad la columna **S14** de cada pregunta con * ***(cumplido con evidencia)***, *◐ (parcial), * ***(no cumplido) o*** ***n/a** (no aplica, con motivo) y, en «Evidencia», dónde se comprueba (por ejemplo, «Plan de datos» o «EDA»). Las columnas S15 y S16 se dejan para esos sprints.
   - **Parte B (clasificación normativa).** Contestad las seis preguntas: datos personales y de qué tipo; base legal; si haría falta una evaluación de impacto (EIPD); **nivel de riesgo del AI Act** (inaceptable, alto, limitado o mínimo); obligaciones que se derivan; y si el sistema apoya decisiones sobre personas. La nota de orientación al final de la parte B os ayuda.
   - **Parte C (registro de riesgos).** Escribid **al menos 5 riesgos específicos de vuestros datos**. Para cada uno: qué puede ir mal y a quién afecta, tipo (datos, modelo, uso, legal o técnico), **probabilidad (P)** e **impacto (I)** de 1 a 3, nivel = P × I (1-2 bajo, 3-4 medio, 6-9 alto), medida, responsable y estado. Mirad los ejemplos por brief al final de la parte C, pero **no los copiéis**: ajustadlos con vuestras cifras.
   - **Parte D (cierre).** No se toca: se rellena en el sprint 16.

   Ejemplo de una fila de la parte C (para un proyecto de series, adaptado de los ejemplos de la plantilla):

   ```
   | R-01 | La previsión falla cuando hay un suceso atípico (una crisis) y el cliente planifica mal el personal | datos | 2
   | 3 | 6 | Avisar en el demostrador de que no cubre sucesos extraordinarios y medir el error en los periodos atípicos |
   (responsable) | abierto | |
   ```

   Dejad vacía la última columna («cómo se evaluó la mitigación»): se rellena en el sprint 16.
5. **Repositorio: README (30 minutos).** Copiad `plantilla_README_repositorio.md` como `README.md` **en la raíz** del repositorio (no dentro de `sprint-14`). Sustituid lo que va entre corchetes en: el título y el brief; el apartado 1 (el problema en tres líneas); el 3 (el árbol ya muestra las carpetas `sprint-14/` a `sprint-17/`; hoy solo existe la de este sprint y las demás se crean en su momento); el 4 (**cómo reproducirlo**: descargar los datos y ejecutar los cuadernos en orden; en «Requisitos» escribid la versión de Python que usáis); el 6 (datos y licencias); el 7 (resumen de IA responsable); el 8 (equipo) y el 9 (tablero). La tabla de resultados (apartado 2) y el demostrador (apartado 5) se rellenan en los sprints 15 y 16: dejad los corchetes.
6. **Repositorio: `requirements.txt` (10 minutos).** En un cuaderno de Colab, ejecutad:

   ```
   import pandas, numpy, sklearn, matplotlib

   for modulo, nombre in [(pandas, "pandas"), (numpy, "numpy"), (sklearn, "scikit-learn"), (matplotlib, "matplotlib")]:
       print(f"{nombre}=={modulo.__version__}")
   ```

   Qué hace cada línea:
   - `import` carga las cuatro librerías; añadid las que importen vuestros cuadernos (por ejemplo `scipy` o `seaborn`).
   - El bucle recorre cada pareja (librería, nombre) y escribe `nombre==versión`. Sale una línea por librería, como `pandas==2.3.3` (la versión que tenga vuestro Colab). `scikit-learn` es el nombre de instalación de `sklearn`.

   Copiad las líneas en un archivo `requirements.txt` en la raíz del repositorio (*Add file → Create new file*). También se puede hacer con `pip show nombre` o, en Colab, con `!pip freeze` y buscando el nombre.
7. **Repositorio: commits y uso de IA (10 minutos).** Comprobad en el historial (icono del reloj) que **todas las personas** tienen al menos un *commit*. Usad mensajes con la forma `tipo: descripción`: `docs: añado backlog`, `data: añado plan de datos`. Ningún dato personal ni claves. En el apartado 7 del README **añadid una línea de «Uso de IA»**: qué pedisteis a la IA y qué comprobasteis a mano.

Qué deberíais ver. `backlog_y_sprints.md` con entre 10 y 15 historias y cuatro objetivos de sprint; `riesgos_eticos.md` con la columna S14 rellena, la parte B contestada y cinco filas en la parte C; un `README.md` y un `requirements.txt` en la raíz; un tablero con las historias; y *commits* de todas las personas.

Hecho cuando: los cuatro documentos están escritos, el tablero tiene las historias y otra persona podría descargar los datos y ejecutar los cuadernos solo con el README.

Si algo falla

Veis...	Qué significa	Qué hacer
La tabla del backlog se ve como texto con barras	Falta una barra vertical o una línea en blanco antes de la tabla.	Pulsad **Preview** antes de guardar y comprobad cada fila.
Tenéis 8 historias o 20	La rúbrica pide entre 10 y 15.	Juntad las que sean muy pequeñas o dejad las demás como ideas fuera del backlog.
Los riesgos son «cuidado con los sesgos» o «hay que proteger los datos»	Son genéricos.	Si valen para cualquier proyecto, no valen: poned qué dato concreto, qué cifra y a quién perjudica.
No sabéis qué poner en probabilidad e impacto	Las dos escalas van de 1 (bajo) a 3 (alto).	Puntuad entre todos; el nivel es P × I. Los de nivel 6 a 9 necesitan una medida que se pueda comprobar.
`ModuleNotFoundError: No module named 'shap'` (con otro nombre)	Esa librería no está instalada en la sesión.	Quitadla del `import` si no la usáis; si la usáis, añadid antes una celda con `!pip install shap`.
El historial solo tiene *commits* de una persona	Una sola persona hizo todas las subidas.	Repartid las subidas: cada persona sube o edita al menos un archivo con su cuenta.
El README dice «Python 3.11» pero Colab usa otra versión	El ejemplo de la plantilla puede no coincidir con vuestro entorno.	Escribid la versión que os salga con `import sys; print(sys.version)`.

Paso 7 · Revisión, entrega, review y retro (día 3, 60 minutos)

1. Revisar el trabajo con la rúbrica, por alguien que no escribió esa parte, y anotar el resultado en el README.
2. Subir lo que falte a `sprint-14/` y cada persona su autoevaluación y coevaluación.
3. Enseñar el sprint en la review: 5 minutos por equipo.
4. Hacer la retro (15 minutos) y apuntarla.

En sencillo. Es el paso de **cerrar el sprint**, como repasar la maleta antes de salir de viaje. Primero, una persona que no escribió una parte la revisa con una tabla de cuatro criterios (así se detectan fallos que quien lo escribió ya no ve). Después, se comprueba que todo está subido y que cada persona sube su hoja personal. Luego, cinco minutos para enseñar el trabajo al docente y, por último, una conversación corta sobre cómo habéis trabajado y qué cambiar. No hay código ni nada nuevo que aprender: es comprobar, subir, contar y anotar.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Rúbrica	La tabla con la que se puntúa el reto: cuatro criterios (corrección técnica, reproducibilidad, análisis y comunicación al cliente, IA responsable), cada uno de 1 a 4.
Autoevaluación	Cada persona puntúa su propio trabajo con la rúbrica.
Coevaluación	Valoráis el trabajo de otra persona o equipo, con respeto y sin juzgar a la persona.
Review	La reunión en la que cada equipo enseña lo que ha hecho al docente (el cliente).
Retro (retrospectiva)	La conversación del equipo sobre cómo ha ido el trabajo: qué seguir, qué cambiar y qué acción concreta tomar.
Raíz del repositorio	La carpeta principal, la que se ve al abrir el repositorio, antes de entrar en `sprint-14`.

Cómo se hace

1. **La revisión con la rúbrica (10 minutos).** Quien revisa **no escribió** esa parte y sigue los cinco pasos de «Cómo se revisa» de entrega y rúbrica:
   - **Ejecutar** lo que dice el README: descargar los datos y ejecutar `01_eda.ipynb` y `02_baseline.ipynb` en orden, en un Colab limpio. Anotar si algo falla o si la métrica del baseline no sale igual.
   - **Leer la ficha como si fuera el cliente:** ¿se entiende la pregunta, la decisión que apoya y cuándo habrá éxito?
   - **Comprobar datos y riesgos:** abrir el enlace de cada fuente, leer la licencia y ver que coincide con el plan de datos; leer los cinco riesgos y ver si valdrían para cualquier proyecto (si sí, son genéricos).
   - **Poner un nivel de 1 a 4 en cada criterio, con una frase de prueba.** Se elige el nivel cuya descripción en la rúbrica **se cumple entera**; si se cumple a medias, es el de abajo. Sin prueba escrita, el nivel no vale. Ejemplos: «3 · la métrica se escribió antes, pero falta el informe de duplicados»; «2 · el README no dice cómo descargar los datos».
   - **Apuntar el resultado** en la sección «Revisión con la rúbrica» del final del `README.md` (ya viene en la plantilla) y arreglar lo más fácil. Lo demás queda como «lo primero que mejoraríamos».

```
**Revisado por:** Lucía · **Fecha:** (dd/mm/aaaa)

| Criterio | Nivel (1–4) | Prueba: una frase de lo que has comprobado |
|---|---|---|
| Corrección técnica | 3 | Los dos cuadernos se ejecutan; la métrica se definió antes, pero falta el informe de duplicados
|
| Reproducibilidad | 2 | El README no dice cómo descargar los datos |
| Análisis y comunicación al cliente | | |
| IA responsable | | |

**Lo primero que mejoraríamos:** añadir al README el enlace de descarga de los datos
```

   Es un ejemplo con dos criterios rellenos (nombre y frases inventados): en el vuestro van los **cuatro**.

2. **Subir lo que falte (15 minutos).** Con la guía de entrega. Comprobad esta lista y **subid en subidas hechas por personas distintas** (cada subida es un *commit* con su nombre):

Fichero	Dónde
`ficha_proyecto.md`, `plan_datos_licencias.md`, `01_eda.ipynb`, `02_baseline.ipynb`, `backlog_y_sprints.md`, `riesgos_eticos.md`	carpeta `sprint-14/` (los seis del equipo)
`README.md` y `requirements.txt`	**raíz** del repositorio, no en `sprint-14/`
`autoevaluacion_nombre.md`	carpeta `sprint-14/`, **uno por persona**

   Los cuadernos de Colab se descargan con *Archivo → Descargar → Descargar .ipynb* (después de *Reiniciar y ejecutar todo*). Los nombres, en minúsculas y sin espacios ni tildes. Si no subís los datos, el `descargar_datos.py` (o el enlace de descarga en el README) permite repetirlos.

3. **La autoevaluación y la coevaluación (10 minutos).** Cada persona copia la hoja autoevaluación y coevaluación con su nombre (`autoevaluacion_marta.md`, por ejemplo) y la sube a `sprint-14/`. Rellena la **Parte 1** (nueve preguntas de repaso con Sí / No / A medias, las cuatro puntuaciones de 1 a 4 con «por qué» y «qué haría para subir un punto», y dos frases sobre lo que más costó y el uso de IA) y la **Parte 2** (coevaluación del trabajo de la persona o equipo que el docente os indique en la review). Cada cual sube lo suyo: no escribáis en el archivo de otra persona.

4. **La review (5 minutos por equipo).** Contad, por este orden: (1) la pregunta de negocio y la decisión que apoyará; (2) los datos y **un hallazgo del EDA que cambia el enfoque**; (3) el baseline y su métrica; (4) tres riesgos éticos; (5) el backlog de los próximos sprints. Repartid las partes entre personas. Los cambios de alcance después del *go* se negocian aquí con el docente.

5. **La retro (15 minutos).** Hablad de tres cosas: una que **seguir haciendo**, una que **cambiar** y **una acción concreta** para el sprint 15 («cada persona sube su trabajo al terminar cada día», no «trabajar mejor»). Anotadlas en la tabla del apartado 5 de `backlog_y_sprints.md`, fila del sprint 14 (la última columna, «¿se cumplió la acción anterior?», lleva `–`). En las retros siguientes, empezad comprobando si cumplisteis la acción anterior. Subid el archivo editado (icono del lápiz → **Commit changes**).

Qué deberíais ver. En `sprint-14/` del repositorio, los seis ficheros del equipo y una hoja de autoevaluación por persona (**6 + N**); en la raíz, el README con la sección de revisión rellena y el `requirements.txt`; y en el historial (el icono del reloj), *commits* de todas las personas.

Hecho cuando: la sección «Revisión con la rúbrica» está rellena, todos los ficheros están subidos, cada persona ha subido su autoevaluación y coevaluación y habéis hecho la review y la retro.

Si algo falla

Veis…	Qué significa	Qué hacer
No sabéis qué nivel poner en un criterio	La rúbrica describe cuatro niveles por criterio.	Leed la fila en entrega y rúbrica; elegid el nivel cuyo texto se cumple entero y escribid la prueba.
Al ejecutar el cuaderno en un Colab limpio falla	Falta un archivo, una librería o el cuaderno depende de una celda que no está.	Anotadlo como hallazgo, corregid lo más fácil y repetid *Reiniciar y ejecutar todo*.
Subisteis `README.md` a `sprint-14/`	Debe estar en la raíz.	Abrid el archivo → icono del lápiz → borrad `sprint-14/` de la ruta de arriba → **Commit changes**.
`autoevaluacion_nombre.md` se ha sobrescrito	Dos personas usaron el mismo nombre de archivo y la segunda borró lo de la primera.	Usad un nombre distinto por persona (nombre propio, minúsculas, sin tildes) y rellenad de nuevo.
Los números de la tabla de la autoevaluación se han descolocado	Se borró alguna barra vertical de la tabla.	Pulsad **Preview** y comprobad que cada fila tiene todas sus barras.
Solo hay *commits* de una persona	El resto no ha subido nada con su cuenta.	Repartid las subidas que quedan: cada persona sube o edita al menos un archivo.
En la retro surgen quejas sin salida	Falta una acción concreta.	Convertid la queja en una acción con quién y cuándo, y apuntadla en el apartado 5 del backlog.

Ayuda 1 · Cómo usar la IA en este reto

En un proyecto largo la IA puede ayudar mucho, pero **la responsabilidad es vuestra** y **puede inventar fuentes, licencias y enlaces**. Reglas:

- **Nunca inventéis una fuente o una licencia.** Si la IA os da un enlace, entrad y comprobadlo.
- **Que el proyecto sea vuestro:** la idea, la pregunta y la métrica salen del equipo. No se acepta copiar un cuaderno de internet.
- **Declarad el uso de IA** en el apartado 7 del README: qué pedisteis y qué comprobasteis a mano.
- **Nada de datos personales ni claves** en una IA ni en el repositorio.

Para qué	Ejemplo de petición	Qué comprobáis
Encontrar fuentes de datos	«Dame fuentes abiertas de datos de llegada de turistas a Canarias y qué licencias suelen tener»	**Entrad vosotros en la web**, leed la licencia y comprobad que los datos existen

Para qué	Ejemplo de petición	Qué comprobáis
Redactar la ficha	«Ayúdame a convertir estas notas en una pregunta de negocio clara con una métrica»	Que sea **vuestra** idea, no una genérica
Planificar	«Convierte este objetivo en 5 historias de usuario pequeñas»	Que son realizables en el tiempo que tenéis
Revisar riesgos	«Para un proyecto de [vuestro brief], ¿qué riesgos éticos y de protección de datos debería considerar?»	Que se aplican a **vuestro** caso y que vuestras medidas son reales

Ayuda 2 · Errores frecuentes (y cómo evitarlos)

Error	Por qué ocurre	Cómo evitarlo
Elegir un problema demasiado grande	Ambición	Preguntaos: ¿se puede hacer en unas 54 horas de proyecto?
Elegir datos sin mirar la licencia	Se quiere avanzar	La licencia es lo primero que se comprueba
Definir la métrica después de ver los resultados	«Para que salga bien»	Escribidla en la ficha y no la cambiéis
Baseline demasiado complicado	Se quiere lucir	El baseline es **lo más simple posible**
Mezclar pasado y futuro al partir los datos	Se parte al azar una serie de tiempo	Con series, partid **por fechas**: entrenad con el pasado y evaluad con el futuro
Backlog con 40 tareas vagas	Se apunta todo	10 a 15 historias concretas con criterio de aceptación
Riesgos éticos genéricos («cuidado con los sesgos»)	Prisa	Cinco riesgos específicos de **vuestros** datos; si valen para cualquier proyecto, no valen
Un solo *commit* del responsable técnico	Es más rápido	Todas las personas hacen *commits* con su cuenta
Dejar la ficha para el final	«Primero los datos»	Entregadla antes de las 14:30 del día 1; sin ella, el docente no puede dar el *go*

Ayuda 3 · Si os atascáis

1. **No nos ponemos de acuerdo en el brief:** votad con criterios (datos disponibles, interés, tiempo) y puntuad cada opción de 1 a 5.
2. **No encontramos datos con licencia:** probad primero con las fuentes de la tabla de briefs y consultad al docente **cuanto antes el día 1**, no por la tarde.
3. **El *go* / *no-go* pide cambios:** es normal; corregid la ficha ese mismo día y seguid.
4. **El cuaderno da error:** leed el mensaje completo, probad la celda con un trozo pequeño de los datos y preguntad al equipo y, después, al docente.
5. Más ayuda: Cómo se trabaja un reto.